

CSE 400: Fundamentals of Probability in Computing

Project Scribe Submission

Group No.: S1 G17

Domain: ITS

Date: March 22, 2026

Scribe Question 1: Baseline System and Motivation for Randomization

Briefly restate the deterministic baseline implemented in Milestone-3. Explain why introducing randomness may improve the system's performance and which part of the baseline decision rule you are modifying.

Answer:

In Milestone 3 we implemented two deterministic baseline controllers for traffic signal control at a four-way intersection using SUMO traffic simulator. Vehicle arrivals follow a Non-Homogeneous Poisson Process with time-varying arrival rate $\lambda(t)$ at three demand phases: build-up ($\lambda = 0.42$ vehicles/s) off-peak ($\lambda = 0.17$ vehicle/s) and peak hour ($\lambda = 0.50$ vehicles/s).

Algorithm 1: Fixed-Timer Controller. This controller acts on an 8-phase signal of fixed timer in which each phase is assigned a predetermined green duration that is not affected by actual traffic conditions. The cycle repeats itself deterministically on the basis of elapsed time only.

Algorithm 2: Time-of-Day (ToD) Controller. This controller changes the duration of green time depending on pre-assigned schedules for each demand phase (15s off peak, 25s build up, 40s peak hour) While this readies itself for time of day, it does not respond to the actual queue conditions at real time within each phase.

Both of these strategies are deterministic, in the sense that signal timing decisions are predetermined, and do not react to instantaneous measurements of traffic using lane-area detectors.

Introducing randomness for the critical failures in deterministic control First of all, fixed timers don't work with stochastic demand. Queue spikes occur at peak times since the controller is not able to respond to burst arrivals. Chernoff bounds confirm that burst events (arrivals exceeding $(1 + \delta)\mu$) occur with non-negligible probability under Poisson demand. Secondly, ToD scheduling was actually worse, with mean waiting time to increase to 648secs vs 447secs for fixed-timer (44.8% degradation). This counterintuitive result shows that to reduce congestion by increasing the cycle times during peaks, without feedback from the actual arrival, worsens congestion when the actual arrivals are different from historical average. Third, deterministic approaches mathematically under-estimate delay because of the Jensen's Inequality: $f(\mathbb{E}[Q]) < \mathbb{E}[f(Q)]$ for convex delay functions.

We are making a modification to the whole signal control decision mechanism. Instead of timer based phase selection, we implement

Algorithm 3: Adaptive Probabilistic Controller that combines: (1) predictive queue estimates $Q^*(\text{dir}) = Q_{\text{current}}(\text{dir}) + \lambda(t) \times \Delta t/4$ (2) a dynamic congestion threshold $T(t) = \lceil \lambda(t) \times 3 \times 4 \rceil$ that in peaks. (3) seven priority ordered green phase rules. (4) a four tier direction selection strategy with starvation rescue guarantees.

Scribe Question 2: Randomized Algorithm Design

Describe the randomized algorithm you have designed. What decision component now involves randomness? What probability distribution or sampling mechanism is used? How does this randomized rule differ from the deterministic baseline? Provide a clear explanation of the algorithm.

Answer:

We have designed Algorithm 3: Adaptive Probabilistic Controller, which is a threshold gated probabilistic signal controller that will work every $\Delta t = 3$ seconds. This is essentially different from straightforward queue-weighted randomization.

Core Algorithm Structure

Every 3 seconds, the controller executes four sequential stages:

Stage 1: Live Detector Reading. The controller reads current vehicle counts from all 8 lane-area detectors (2 per direction: N, S, E, W) installed at the intersection via SUMO detector system.

Stage 2: Predictive Queue Estimation. Rather than using raw detector counts, we compute a predictive estimate that anticipates arrivals over the next decision interval:

$$Q^*(\text{dir}) = Q_{\text{current}}(\text{dir}) + \frac{\lambda(t) \times \Delta t}{4}$$

The division by 4 accounts for four directions sharing the arrival stream. This forward-looking estimate is critical—it prevents reactive lag where the controller switches only after congestion has already built.

Stage 3: Dynamic Threshold Computation.

$$T(t) = \lceil \lambda(t) \times \Delta t \times \text{THRESHOLD_K} \rceil = \lceil \lambda(t) \times 3 \times 4 \rceil$$

During off-peak ($\lambda = 0.17$), $T(t) \approx 3$ vehicles. During peak ($\lambda = 0.50$), $T(t) \approx 6$ vehicles. During high demand, without the manual tuning the automatic tightening makes the controller more reactive.

Stage 4: Decision Logic. The algorithm applies two main decision hierarchies.

Seven Green Phase Rules (Priority Order)

The controller first determines whether to hold current green phase or to change to a new direction:

- (a) **G_MAX Rule:** IF `phase_timer` ≥ 45 s then force switch. Doesn't allow any direction to monopolize the signal for an indefinite period of time.
- (b) **G_MIN Rule:** IF `phase_timer` < 8 s (peak) or < 3 s (off-peak) then hold green. To prevent thrashing between phases, minimum service guarantee is ensured.
- (c) **All Empty Rule:** IF total queue in all directions = 0 then it give green light. Nothing to serve elsewhere, no benefit to switching.
- (d) **Congestion Rule:** IF $Q^*(\text{current direction}) \geq T(t)$ then hold green. Switch lanes to finish serving the clogged lane.

- (e) **Urgent Cut Rule:** IF there is another direction which has $Q^* \geq T(t)$ then switch on the fly Preempt current phase to serve urgent congestion in other places.
- (f) **Drain Done Rule:** IF current lane was empty and others was waiting in queue to enter lane urge to switch. Clean handoff with no waste of timer cycles.
- (g) **Drain Cap Rule:** IF `phase_timer` $\geq 2 \times \text{G_MIN}$ then rotate. Prevents starvation on low traffic lanes by causing it to periodically choose.

Four-Priority Direction Selection

The green phase rules call for a switch, the controller selects the next direction to be switched by means of a four level priority chain:

- (a) **SOLE:** IF only one lane has vehicles then demand on its own, send it directly. Deterministic selection with an obvious choice.
- (b) **URGENT:** IF any lane has $Q^* \geq T(t)$ then it serve the most congested lane. We use deterministic priority to avoid the queue explosion.
- (c) **STARVE RESCUE:** IF any lane has been skipped ≥ 2 times $\rightarrow$ guarantee service which also prevents the probabilistic starvation.
- (d) **Lottery:** ELSE $\rightarrow$ weighted random selection with starvation boost.

Probability Distribution in Lottery

The lottery (tier 4) uses a Categorical Distribution with weights:

$$w_i = Q_i^* + \epsilon + \text{boost}_i$$

where boost_i is the starvation boost that doubles exponentially for lanes skipped more than `MAX_SKIP` = 2 times:

$$\text{boost}_i = \begin{cases} 0 & \text{if skip}_i \leq 2 \\ 2^{(\text{skip}_i - 2)} \times q_{\text{max}} & \text{if skip}_i > 2 \end{cases}$$

Normalized probabilities become $P(i) = w_i / \sum_j w_j$, and selection uses inverse CDF sampling via NumPy's `np.random.choice()`.

Key Differences from Deterministic Baseline

- Uses predictive estimate Q^* as opposed to clock time.
- Dynamic threshold $T(t)$ adapts to arrival rate $\lambda(t)$
- Seven conditional rules instead of fixed timer.
- Probabilistic tier guarantees fairness and congestion priority.
- Starvation rescue guarantees service in 2 rotation cycles.

Scribe Question 3: Probabilistic Reasoning Behind the Algorithm

Explain the probabilistic reasoning that motivates your randomized approach. How does randomness help address uncertainty in the system? What probabilistic assumptions or models support your design? Why is this randomized strategy expected to improve performance?

Answer:

How Randomness Addresses System Uncertainty

Traffic arrivals follow a Non-Homogeneous Poisson Process, that is, there is fundamental stochastic uncertainty:

$$A(t) \sim \text{Poisson}(\lambda(t) \cdot \Delta t)$$

This uncertainty manifests in three critical ways:

Burst Arrival Events. Using the Chernoff bound for Poisson random variables:

$$P(X \geq (1 + \delta)\mu) \leq e^{-\mu\delta^2/3}$$

During peak hour with $\lambda\Delta t = 1.5$, the probability of arrivals exceeding the $2\times$ the mean is approximately 14.3%. The tail event occurs frequently enough that deterministic timers which allocates green phase based on average demand systematically fails during the bursts.

Queue Spikes from Poisson Variance. Queue evolution follows:

$$Q(t+1) = \max(Q(t) + A(t) - D(t), 0)$$

Since $\text{Var}(A) = \lambda\Delta t$, variance of queue accumulates with time. Deterministic controllers have no means to follow instantaneous spikes because their decision rule relies solely on the clock time but not on the observed queue state.

Nonlinear Delay via Jensen's Inequality. Traffic delay is a convex function of queue length. For any convex f :

$$f(\mathbb{E}[Q]) < \mathbb{E}[f(Q)]$$

This verifies the fact that average-based fixed timers systematically pump delay. Real Traffic Delay is higher than predicted with average queue values, which validates the need for controllers that track real-time queue deviation.

Probabilistic Assumptions and Models

Poisson Arrival Process. We assume independent arrival at each direction with rate $\lambda(t)$ which varies with time of day phase. This was tested to give an empirical estimate of arrival counts against theoretical Poisson distributions using Chi-squared goodness-of-fit tests in M3 simulations.

Markov Property of Congestion State. We define state of binary congestion state $S(t) = \mathbb{1}[Q(t) > T(t)]$. State transitions depend neither

on history nor on past state. This makes it possible to model the system as designating a Markov Decision Process and calculate transition probabilities P_{11} (stays congested) and P_{10} (escapes congestion).

Independence Across Directions. We take it to be independent of each other arrivals from orthogonal directions (N, S, E, W). This simplifies the calculation of the joint probabilities, but can be violated if upstream signals induce correlated platooning as a limitation we admit.

Why Randomization Improves Performance

Chebyshev Bounds on Queue Extremes. Applied to queue lengths:

$$P(|Q - \mu| \geq k\sigma) \leq \frac{1}{k^2}$$

With $k = 2$, there is at least 75% probability that queue deviations exceed 2σ from the mean. The reactive controller tracks the deviations.

Threshold-Gated Response to Tail Events. The dynamic threshold $T(t)$ acts as a real-time anomaly detector. When $Q^* \geq T(t)$, When when, the Urgent Cut rule is executed immediately before the current phase.

Starvation Rescue vs Pure Weighted Lottery. A purely probabilistic $P(i) \propto Q_i$ lottery can cause starvation of the low traffic lanes for an indefinite time during peak hours. Our exponential boost $2^{(\text{skip}-2)} \times q_{\max}$ ensures that after 2 skips the boost is dominating the base queue weight ensuring selection. This hybrid deterministic-probabilistic approach is fair with no loss of efficiency.

Predictive vs Reactive Control. The formula $Q^*(dir) = Q_{\text{current}} + \lambda(t)\Delta t/4$ is forward looking in nature. It anticipates arrivals during the next decision approach and does not respond to the current state only. This predictive aspect of the control minimizes the lag of control made by the controller before congestion fully materializes instead of after queues already have grown

Ergodicity and Congestion Escape. The stochastic lottery guarantees that all the congestion states (combinations of per-direction queues) will reach all other states with some non-zero probability. This prevents the occurrence of absorbing states in which some of the lanes are permanently jammed. This measured 3.4×improvement in P_{10} (escaping congestion) validates this theoretical advantage.

Scribe Question 4: Implementation within the Simulation Pipeline

Describe how the randomized algorithm has been integrated into your existing simulation pipeline. What changes were made to the code or system architecture? How is randomness generated or sampled in the implementation? Does the rest of the pipeline remain the same as the baseline?

Answer:

The Adaptive Probabilistic Controller integrates into our Python-SUMO-TraCI simulation pipeline through a single modified control file.

Simulation Architecture

The system consists of four components:

Demand Generator (`src/generate_demand.py`). Generates `data/demand.rou.xml` with Poisson-distributed vehicle arrivals using time-varying $\lambda(t)$. This file is common to the 3 controller simulations (Fixed, ToD, Adaptive) in order to have the same sequence of arrivals for a fair comparison of the results.

SUMO Traffic Simulator. Deals with microscopic dynamics of vehicles, such as acceleration, car-following (Krauss model) and physical movement through the intersection network described in `data/intersection.net.xml`.

TraCI Interface. Allows communication between Python control scripts and SUMO in real-time, respectively. Every 3 seconds, TraCI reads lane area (e2 detector counts in `data/sensors.add.xml`) and receives signal phase commands from the controller.

Decision Controllers. Three separate Python scripts implement the three algorithms:

- `src/runner.py` — Algorithm 1 (Fixed-Timer)
- `src/runner_tod.py` — Algorithm 2 (Time-of-Day)
- `src/runner_random.py` — Algorithm 3 (Adaptive Probabilistic)

Code Modifications

The ONLY modification is the control logic in `runner_random.py`. All other components (SUMO network, detectors, arrival generation, metrics logging) remain identical to M3.

Deterministic Baseline (M3):

```
# Fixed-Timer (runner.py)
def get_next_phase(current_time):
    return (current_time // PHASE_DURATION) % NUM_PHASES

# Time-of-Day (runner_tod.py)
def get_green_duration(sim_time):
    """Return green duration (seconds) based on time-of-day
    demand phase."""
    if sim_time < 50:
        return 15 # Off-peak = 0.17 ~200 veh/hr
    elif sim_time < 150:
```

```

        return 25      # Build-up   =0.42   ~500 veh/hr
    else:
        return 40      # Peak hour  =0.50   ~600 veh/hr

```

Adaptive Probabilistic (M4):

```

import numpy as np

def predictive_queue(q_current, lam):
    return q_current + (lam * DELTA_T) / 4.0

def get_threshold(t):
    return math.ceil(get_lambda(t) * DELTA_T * THRESHOLD_K)

def pick_next_direction(live_q_star,
                        threshold_now, skip_counts, current_dir):

    eligible = [
        d for d in DIRECTIONS
        if not should_skip_empty(live_q_star[d], skip_counts[d])
    ]
    if len(eligible) == 0:
        eligible = [current_dir]

    # Priority 1: SOLE
    non_empty = [d for d in eligible if live_q_star[d] >= 0.1]

    if len(non_empty) == 1:
        chosen = non_empty[0]
        print(f" SOLE {chosen} Q*={live_q_star[chosen]:.1f} |
              direct serve")
        return chosen

    # Priority 2: URGENT
    urgent = [d for d in eligible if live_q_star[d] >= threshold_now]
    if urgent:
        chosen = max(urgent, key=lambda d: live_q_star[d])
        print(f" URGENT {chosen} Q*={live_q_star[chosen]:.1f} >=
              T={threshold_now}")
        return chosen

    # Priority 3: STARVE RESCUE
    starving = [d for d in eligible if skip_counts[d] > MAX_SKIP]
    if starving:
        chosen = max(starving, key=lambda d: skip_counts[d])
        print(f" STARVE RESCUE {chosen} skip={skip_counts[chosen]}")
        return chosen

    q_max_e = max(live_q_star[d] for d in eligible) or 1.0

    # Priority 4: Weighted Lottery
    weights = []

```



```
for d in eligible:
    boost = 0.0
    if skip_counts[d] > MAX_SKIP:
        extra = skip_counts[d] - MAX_SKIP
        boost = (2 ** extra) * q_max_e
    weights.append(live_q_star[d] + EPSILON + boost)
weights = np.array(weights, dtype=float)
probs = weights / weights.sum()
return np.random.choice(eligible, p=probs)
```

Random Number Generation

We use NumPy's Mersenne Twister (MT19937) with seeded initialization:

```
np.random.seed(42)
```

The system has two independent randomness layers:

- (a) **Arrival Layer:** Poisson vehicle generation in `generate_demand.py`
- (b) **Control Layer:** Lottery direction selection in `runner_random.py`

Pipeline Consistency

The simulation patterns which remains same:

- Same 4 way intersection topology with 8 lane-area detectors
- There is same poisson arrival rates i.e. 0.17 / 0.42 / 0.50 vehicles/s across phases.
- Same SUMO physics engine (Krauss car-following model)
- Same metrics collection: queue length, waiting time, congestion transitions
- The simulation time is also same i.e. 300 secs

That way, all observed differences in performance are attributable only to the control strategy, and not confounding changes in the simulation setup or other changes in traffic demand.

Scribe Question 5: Evaluation Metrics and Comparative Results

Identify the evaluation metrics used to measure system performance. Why are these metrics appropriate for your problem domain? How does the randomized algorithm perform compared to the deterministic baseline? Present and interpret the results of this comparison.

Answer:**Evaluation Metrics**

We measured six performance metrics across all three controllers:

- (a) **Mean Waiting Time:** Average time that vehicles spend in the system from arrival to departure. It directly affects cost and driver experience for example travel time reliability, consumption of fuel, emissions etc. .
- (b) **Peak Waiting Time:** Delay that any vehicle experiences the most. Catch worst case tail events that are important for safety and driver frustration.
- (c) **Mean Queue Length:** Average vehicles queuing per direction. Primary control objective as it relates to delay using Little's Law.
- (d) **Queue Variance:** Measures system stability and predictability of service.
- (e) **Congestion Persistence (P_{11}):** Markov transition probability of being congested again, if currently congested Lower is better.
- (f) **Queue Clearing Speed (P_{10}):** Markov transition probability of escaping congestion if congested. Higher is Better-higher is the decongesting capability.

These are common measures in traffic engineering, and are directly tied to practical considerations: costs of operations, safety of drivers, space needed on the road, and reliability of the system.

Comparative Results

We ran each algorithm for 300 seconds under identical Poisson arrival demand:

Metric	Fixed Timer	Time-of-Day	Randomized Adaptive
Mean Wait (s)	447	648	281
Peak Wait (s)	1568	2896	1039
Mean Queue	3.01	3.28	2.94
Queue Variance	6.56	6.29	6.67
P_{11} (Stays Congested)	0.896	0.907	0.643
P_{10} (Clears Congestion)	0.104	0.093	0.357
Improvement vs Fixed	baseline	+44.8% worse	-37.2% better

Key Performance Achievements

- **37.2% reduction** in mean waiting time: 447s $\rightarrow$ 281s
- **34% reduction** in peak waiting time: 1568s $\rightarrow$ 1039s
- **3.4 $\times$ faster** congestion clearing: P_{10} increased from 0.104 to 0.357

- **28% less persistent** congestion: P_{11} decreased from 0.896 to 0.643

Critical Findings: Time-of-Day Perform worse than before The ToD controller voided mean wait (44.8% worse than Fixed-Timer) to 648s and peak wait to 2896s (84.7% worse). This counter-intuitive result shows a fundamental flaw in deterministic scheduling:

Why ToD Failed: ToD controller assigns 40 seconds of green during high hours of demand (150-300s) based on historical averages of demand. However, when actual instantaneous arrivals differ from the historical average by the variance of the Poisson arrival process, these long fixed cycles are a trap for the duration of 40 seconds even if, during that time, other directions are congested. The controller has no knowledge of real-time queue accumulation of each phase.

Interpretation

Threshold-Gated Control Outperforms Pure Scheduling. The dynamic threshold $T(t)$ enables the Adaptive controller to identify the congestion the instant $Q^* \geq T(t)$ and respond instantly through Urgent Cut. Fixed and ToD controllers wait for predetermined timer expirations, by which time queues have already grown.

Predictive Estimates Reduce Control Lag The formula $Q^* = Q_{\text{current}} + \lambda(t)\Delta t/4$ anticipation, Arrivals over next 3 seconds will be 10. This forward looking approach switches signals before the congestion is fully materialized, whereas, reactive controllers do switch only after seeing large queues.

Starvation Rescue Achieves Fairness Without Throughput Loss. Despite the 2.6% queue reduction, no direction experienced starvation. The exponential boost in service ensures service within 2 cycles while the weighted lottery still prioritizes busier lanes. This shows that fairness and efficiency are not in contradiction when stochasticity is suitably gained.

Markov Transition Improvements Ergodicity The 3.4 improvement in P_{10} shows that the system is actively escaping congested states and not staying in them. This confirms the theory making prediction that stochastic control covers the exploration of all the regions of state space, avoiding absorbing states where lanes are forever jammed.

Scribe Question 6: Insights and Remaining Challenges

Based on your experiments: What insights did you gain about the effect of randomness in your system? In what situations does the randomized strategy perform better or worse than the baseline?

Answer:

Key Insights

1. Fixed timers fail under stochastic demand. Queue spikes appear during peak periods because deterministic controllers cannot respond to burst arrivals. Chernoff bounds confirm burst events (arrivals $\geq (1 + \delta)\mu$) occur with 14-20% probability—frequent enough to cause systematic congestion. Our Adaptive controller's Urgent Cut rule responds to these tail events in real time.

2. ToD scheduling better variance but not better peak congestion. Adapting the green durations to the time-of-day reduces the fluctuation of queues marginally ($\sigma^2 = 6.29$ vs 6.56 for Fixed) perhaps but the controller is still blind to real time conditions within each phase. The 44.8% degradation in mean wait time which demonstrates that the longer cycle time during peak value worsen the congestion when there is deviation of arrivals from historical pattern.

3. Adaptive probabilistic control gives 37.2mean wait. By using predictive queue estimate Q^* , dynamic threshold $T(t)$, and starvation-aware direction picker in conjunction, the controller maintains the mean waiting time at 281s instead of the 447s baseline (using fixed timer only). The main innovation is threshold gated probabilistic control which is not only use randomization but also constrained stochasticity with deterministic which overrides the rules.

4. Chebyshev and Chernoff bounds We understand why the deterministic methods do not work The probability of burst arrivals above a fixed threshold is non-negligible under Poisson demand. A reactive controller that follows these deviations as they happen is demonstrably better. Our measured $P_{10} = 0.357 \text{ vs } 0.104$ that confirms the theoretical advantage.

5. Fairness and efficiency are not in conflict The starvation rescue mechanism ensures every lane is served within 2 rotation cycles, whilst the weighted lottery ensures busier lanes are still given priority. No trade-off between fairness and throughput was seen.

When Randomized Performs Better

- **Variance of high arrival:** When queue spikes are caused by Poisson burst traffic, the threshold-gated switching offers immediate queue relief in the presence of fixed timers waiting for the completion of cycles.
- **Asymmetric demand:** In the event that one direction consistently has higher traffic, weighted lottery automatically gives proportionately more green time, without any tuning the system by hand.
- **An Unpredictable tail events:** Special occasions or crises make demand soar which deterministic schedules react to, Predictive estimate Q^* sees the early entrance of these.

- **Moderate to high load ($\rho = 0.5$ to 0.8):** Queues are formed but system is not saturated. Dynamic threshold avoids growth of queues by dynamically prioritizing.

When Randomized Performs Worse or Equivalently

- **Very low load ($\rho < 0.3$):** On-line Adaptive switching adds unnecessary overhead from detector polling.
- **Uniform demand:** Assuming arrivals are deterministic (not Poisson) in which case well-tuned fixed time has the best performance. Stochasticity doesn't give any benefit without uncertainty.
- **Extreme saturation ($\rho > 0.9$):** All lanes are perennially clogged. Queue-weighted lottery degenerates to near-uniform distribution without having strategic benefit. System requires additional capacity lanes.
- **Strong spatial correlation:** If upstream signals generate synchronized platoons, there is the possibility that randomized switching will disrupt platoons and lead to lower throughput. Through movements may be better done with coordinated deterministic timing - (green phase).

Remaining Challenges

- **Parameter sensitivity:** parameters (THRESHOLD_K = 4, MAX_SKIP = 2, G_MIN = 8s) were manually tuned. In future milestones we can include adaptive learning by reinforcement to optimizing the parameters based on observed performance.
- **Network-level coordination:** In the current implementation controls one isolated intersection. Real traffic networks need multi-agents coordination for green wave progression at different signals.
- **Independence assumption violation:** Assumption of independence of arrivals from orthogonal directions, but with upstream signals they produce spatial correlation. Copulas or multivariate Poisson processes are required for models that will be developed in the future.
- **Real-world deployment constraints:** Actual traffic signals: minimum yellow time (3-5s), all red signal clearance interval, provide for pedestrian crossings. Can't change phases every 3 seconds in real life. Extension Meaning Integrating safety constraints.

End of Submission